

Assignment Tracker

System Architecture ->

Frontend ->

Code written in javascript using ReactJS library. Communicates with the backend using AXIOS

Routes ->

1. /signup = signup using credentials for both students and teachers
2. /login = login using credentials for both students and teachers
3. /teacher and /teacher/create – to view submissions and create assignments for students
4. /student and /student/submit to view assignments and submit a assignment
5. Files are currently stored locally in the folder

Backend ->

Code written in python using FAST API library.

1. Routes organized for login , signup , assignment and submission
2. Uses SQLite database for storage of credentials and assignments

Database ->

Using SQLite below tables are created ->

1. User
2. Assignment
3. Submissions

RELATIONSHIP ->

Relationship	Type
Teacher creates Assignment	One-to-Many (1:N)
Student submits Assignment	Many-to-Many (via Submission)
Assignment has Submissions	One-to-Many (1:N)
Student has Submissions	One-to-Many (1:N)

ENTITIES ->

Entity	Attributes
Teacher (USER)	Id , username , password , role
Student (USER)	Id , username , password , role
Assignment	Id , title , description , createdBy , filePath
Submission	Id , content , assignmentId , studentId , filePath

API ENDPOINTS ->

1. Assignment creation by teacher ->

POST [/teacher/create]

Body -> Title , Description , File

2. Assignment submission by student ->

POST [/student/assignments/submit]

Body -> content , File

3. Submissions viewed by teacher ->

GET [/teacher/assignments/assignmentID/submissions]

Authentication Strategy ->

1. Login and Signup are done through /signup and /login
2. Credentials are stored in the SQLite Database only
3. React stores sessions locally after login and backends checks credentials on every route
4. Conditional checks are there so neither students can create assignment nor teachers can submit assignments.

Scalable System ->

To scale the system ,

1. migrate sqlite database to PostgreSQL database
2. Files submitted or uploaded can be stored in AWS or AZURE or any other cloud database
3. JWT authentication can be implemented for better security
4. Integrate Github actions for automated testing and deployment